

General Training for Programming Competitions & Coding Interviews

Winter Semester 2024/2025

Graph Algorithms

Markus Fischer, Paul G. Rump, Christoph Staudt

Friedrich-Schiller-Universität Jena



Maximum Flow & Minimum Cut

MAXIMUM FLOW & MINIMUM CUT

Repetition: The Maximum-Flow problem

Task:

- Given a graph $G = (V, E)$ with edge capacities $c : E \rightarrow \mathbb{R}^+$ (called *flow network*), and
- a pair of nodes $s, t \in V$ (called *source* and *sink*).
- Find a *flow*, that is a mapping $f : E \rightarrow \mathbb{R}^+$ with:

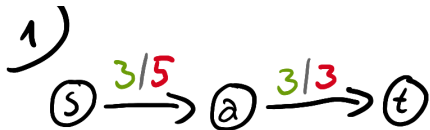
Capacity Constraints: $\forall e \in E : f(e) \leq c(e)$

Conservation of flow: $\forall v \in V \setminus \{s, t\} : \sum_{u \in V : (u, v) \in E} f((u, v)) = \sum_{w \in V : (v, w) \in E} f((v, w))$

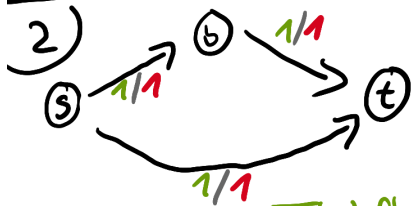
- The *size* of the flow is the amount of flow that leaves s , that is:

$$F = \sum_{v \in V : (s, v) \in E} f((s, v))$$

This is the value we need to maximize.

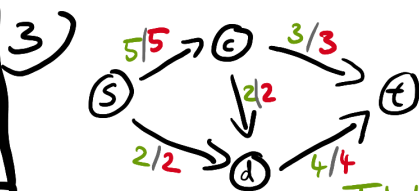


Total flow : 3

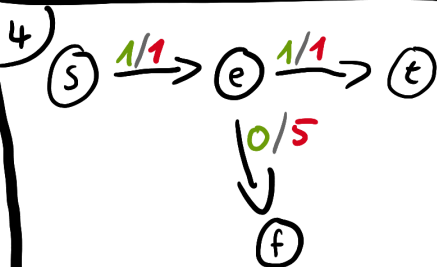


Total flow : 2

Flow of the edge
Capacity of the edge



Total flow: 7



Total flow : 1

Repetition: Algorithm by Edmonds and Karp

- Basic idea: starting from an empty flow, find any valid (s,t) -flow with as few edges as possible. Add this flow to the total flow and adjust the remaining capacities accordingly. Iterate until no additional flow is found.
- “Adjusting the capacities” means: *reducing* the capacities in direction of the flow, and *increasing* the capacities in opposite direction.
- The base algorithm is by Ford and Fulkerson, but that algorithm has suboptimal behavior in special cases.
- The Edmonds-Karp-algorithm has time complexity $\mathcal{O}(|E|^2 \cdot |V|)$.

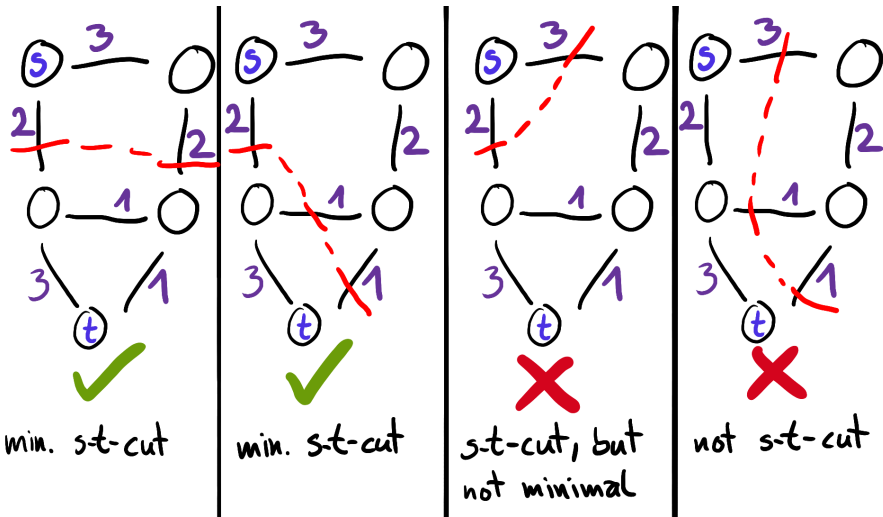
Repetition: The MinCut-Problem

Task:

- Given a graph $G = (V, E)$ with edge weights $c : E \mapsto \mathbb{R}^+$ (a *flow network*), and
- a pair of nodes $s, t \in V$.
- Find a set of edges $E' \subset E$ such that after “cutting” (removing) E' (that is: removing E' from the graph), the remaining graph $G' = (V, E \setminus E')$ is disjunct and s and t are in different components.

Then E' is the *cut-set* of the (s, t) -cut given by the components of G' , and the sum of the weights of the edges in E' is the cost of the cut. E' is further called *minimum* (s, t) -cut, if there is no other (s, t) -cut with smaller costs.

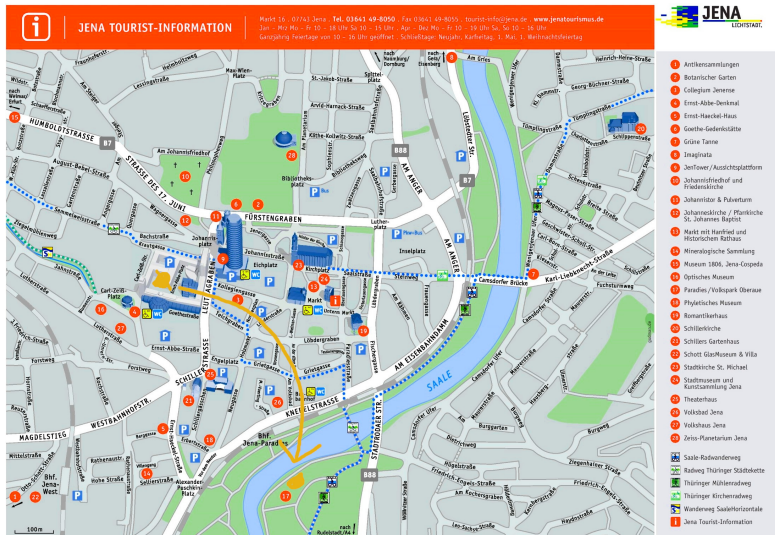
Example of minimum cuts



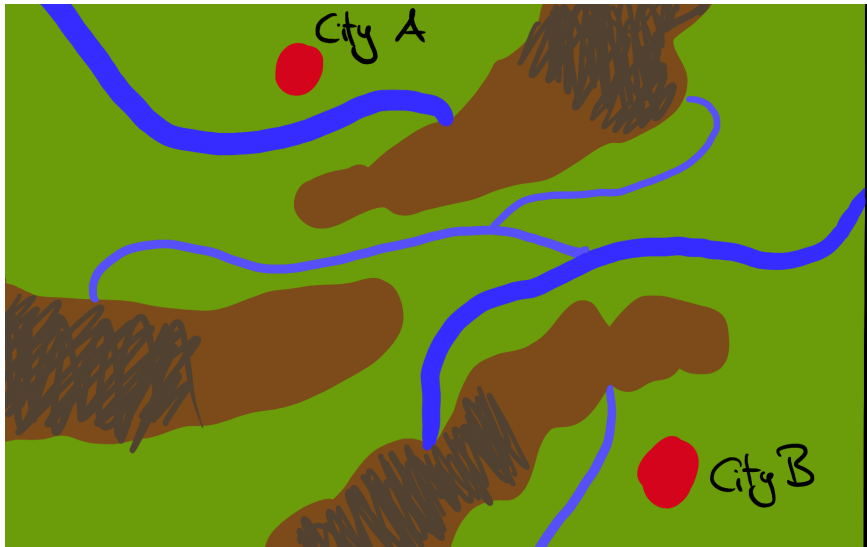
Real-world examples for MinCut-Exercises:

- Walk through a city and cross as few streets as possible.
- Build a railroad trough difficult terrain with minimum cost.

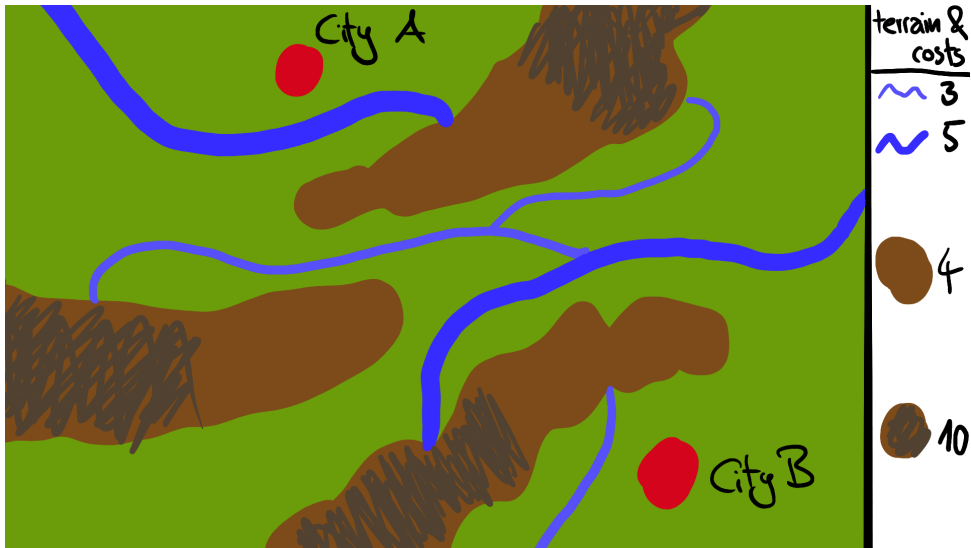
MinCut: examples



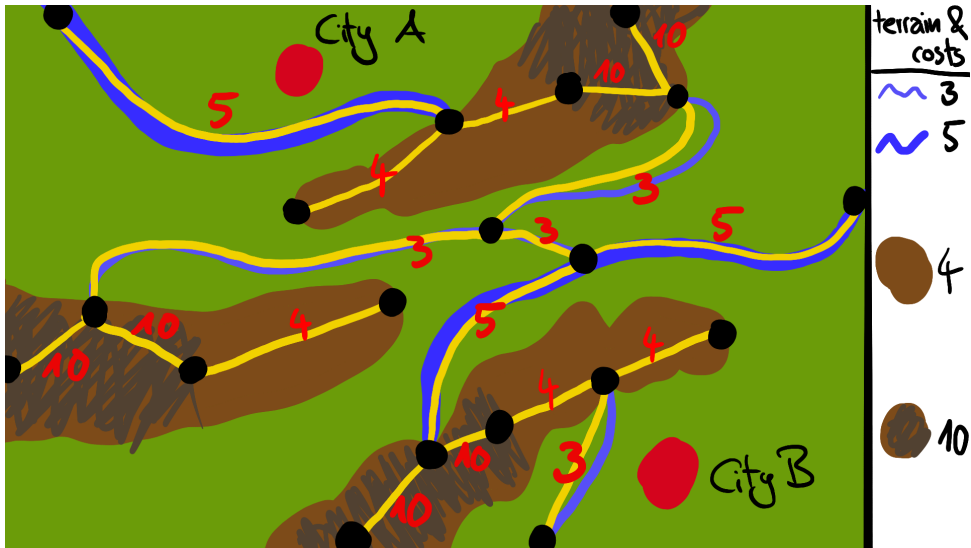
MinCut: examples



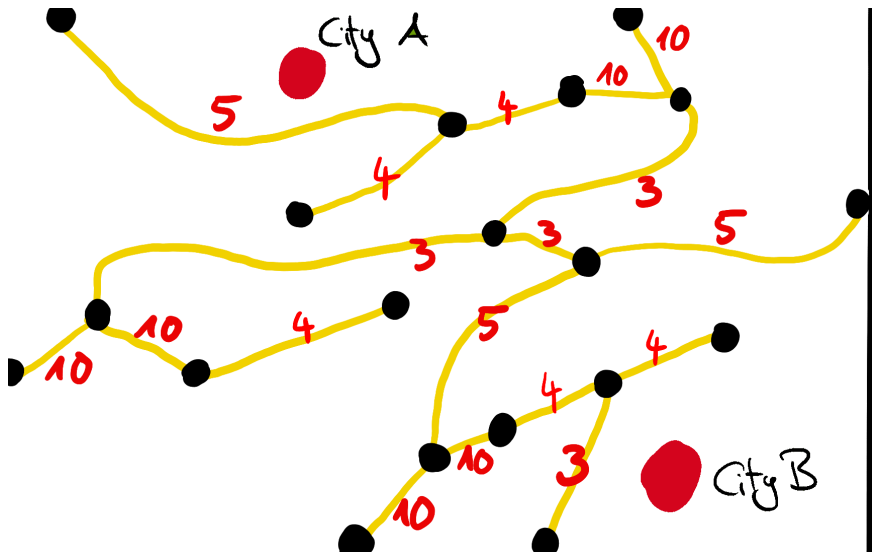
MinCut: examples



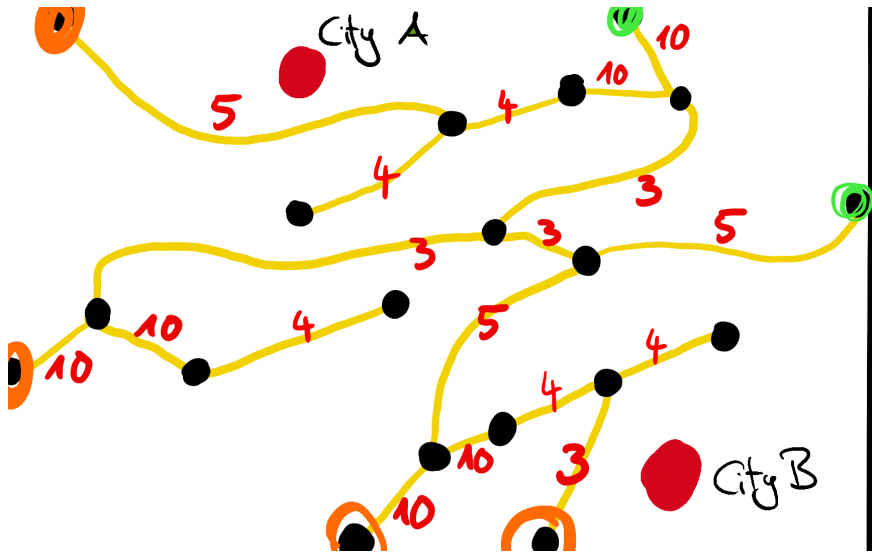
MinCut: examples



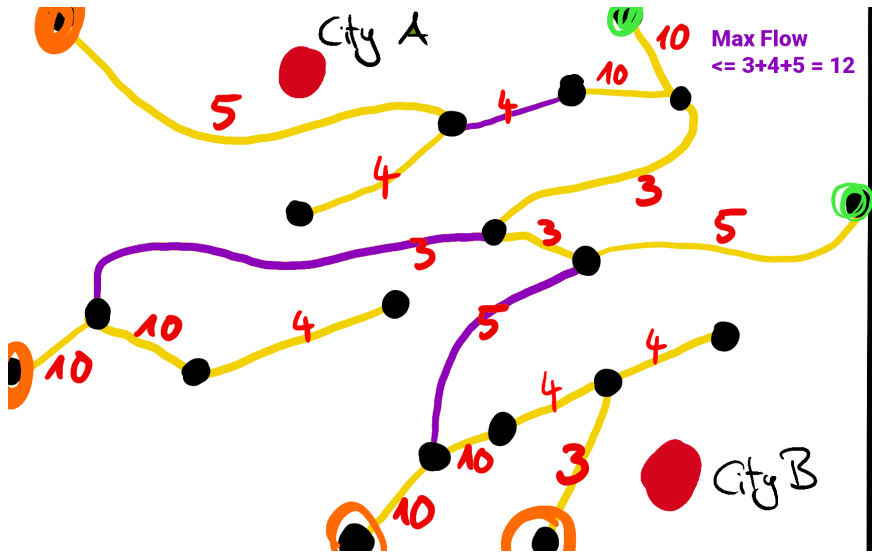
MinCut: examples



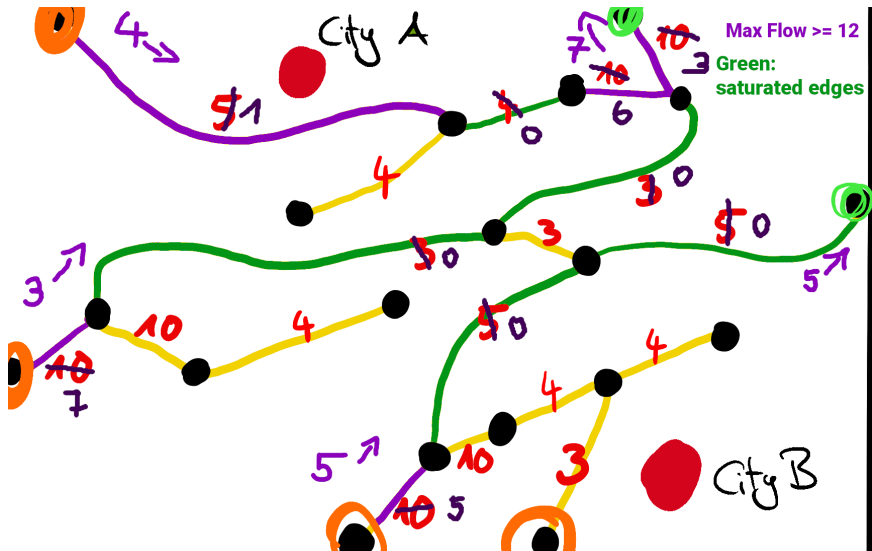
MinCut: examples



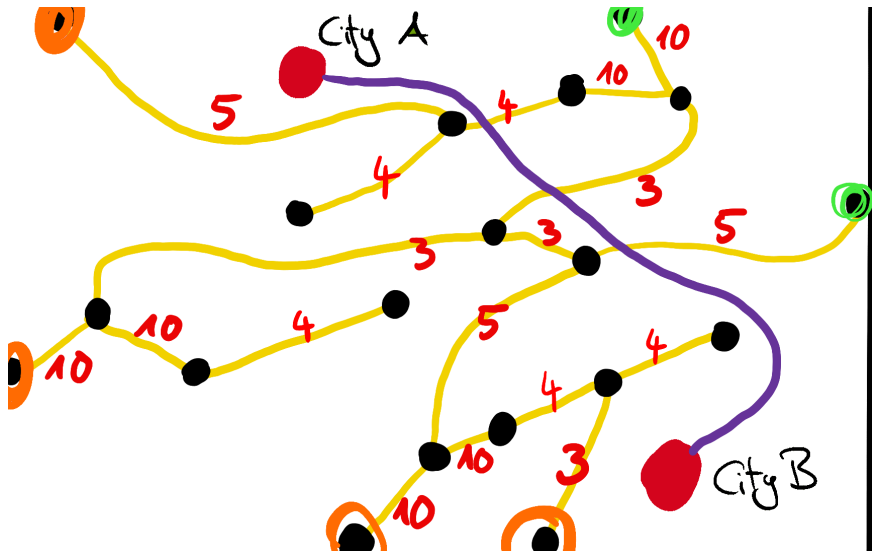
MinCut: examples



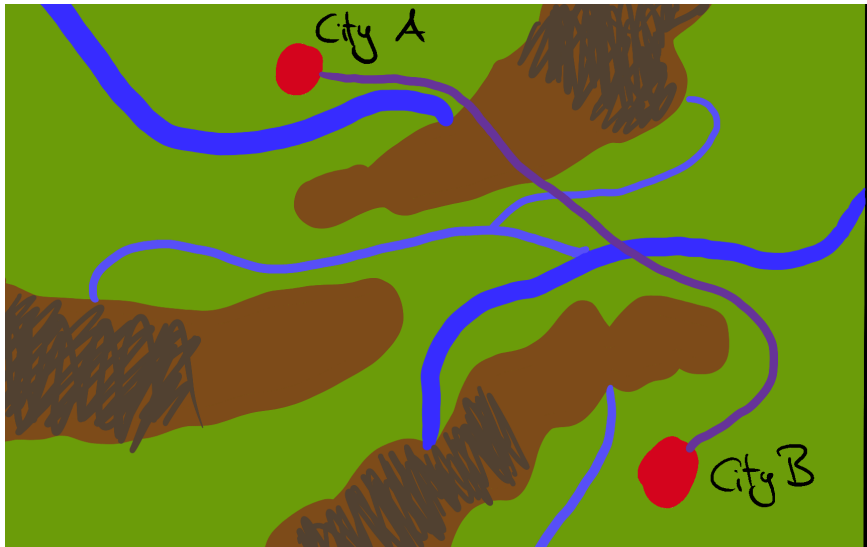
MinCut: examples



MinCut: examples



MinCut: examples



Repetition: The MaxFlow-MinCut theorem

- Given a flow network (G with capacities c and source s and sink t).
- The MaxFlow-MinCut theorem states that if f is a maximum (s,t) -flow, then any minimum cut has also weight f .
- Therefore, we can simply use any MaxFlow-algorithm to find a minimum cut!

Matchings

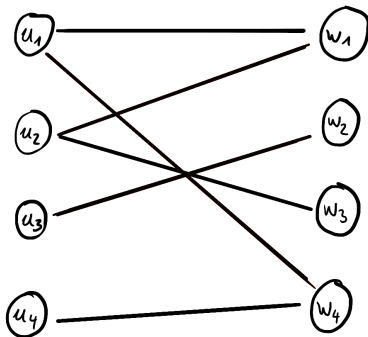
MATCHINGS

Definitions

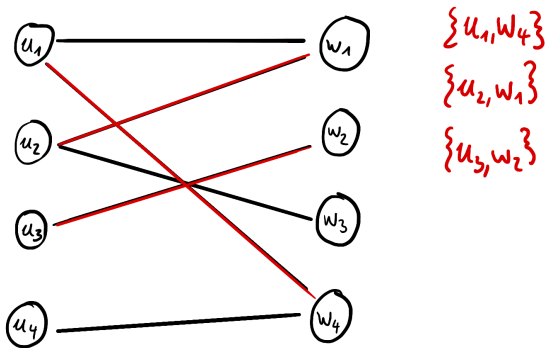
Let $G = (V, E)$ be a bipartite graph, that is $V = U \cup W$ with $U \cap W = \emptyset$ and $E \subset U \times W$.

- A *matching* is a set of pairs of nodes, where each pair of nodes is connected by an edge: but each node belongs to *at most* one pair.
- A matching is called *maximum matching*, if there is no other matching that contains a larger total number of node-pairs.
- A matching is called *perfect*, if each node belongs to exactly one pair (i. e. each node is matched to some other node).

Matchings: Examples

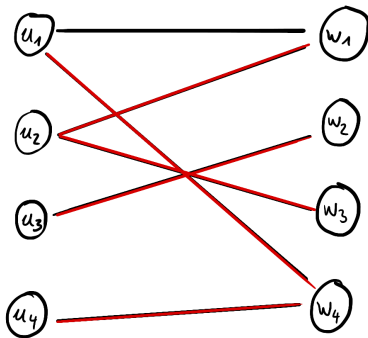


Matchings: Examples



Obviously not a perfect matching. But is it a maximum matching?

Matchings: Examples



$\{u_1, w_4\}$

$\{u_2, w_1\}$

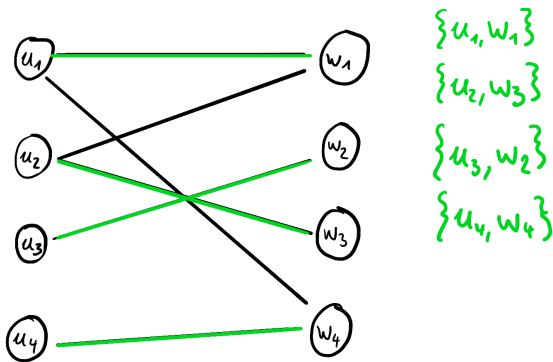
$\{u_3, w_2\}$

$\{u_2, w_3\}$

$\{u_4, w_4\}$

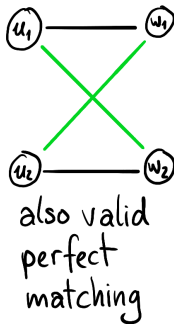
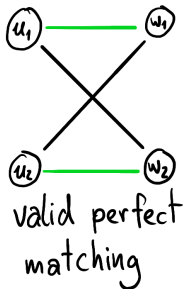
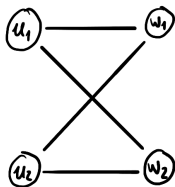
Not a valid matching.

Matchings: Examples



A perfect matching, and therefore also a maximum matching.

Matchings: Examples

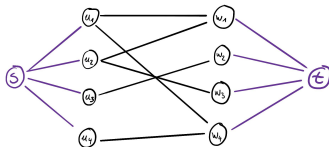
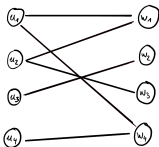


Perfect matchings are not unique!

How to find a maximum matching

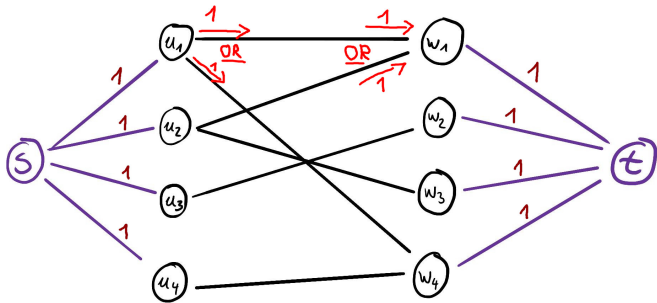
The maximum matching problem can be reduced to the maximum-flow problem by the following construction:

- Add a new node s . Add edges $\{s, u\}$ for every $u \in U$.
- Add a new node t . Add edges $\{w, t\}$ for every $w \in W$.
- Define capacities $c(e) = 1$ for every edge e that is now in the graph.

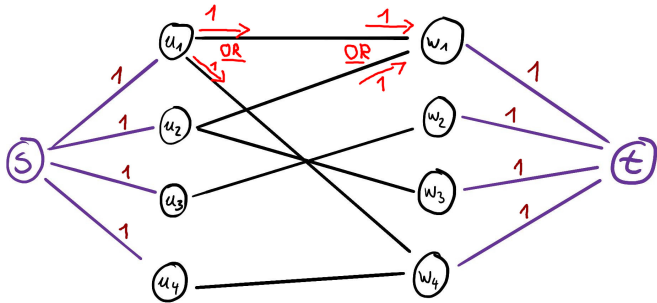


How to find a maximum matching

Now in each valid flow, each node $u \in U$ will have incoming flow 0 or 1 (because it has only one connection to s with capacity 1), and therefore the flow via u will only go through a single $w \in W$ before reaching t . By the same argument, each $w \in W$ has incoming flow from at most one $u \in U$. Therefore, the edges (u, v) of the maximum flow are a valid matching.

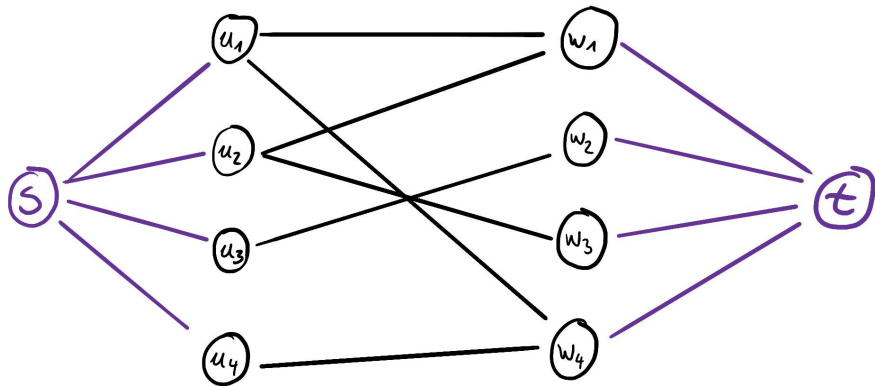


How to find a maximum matching

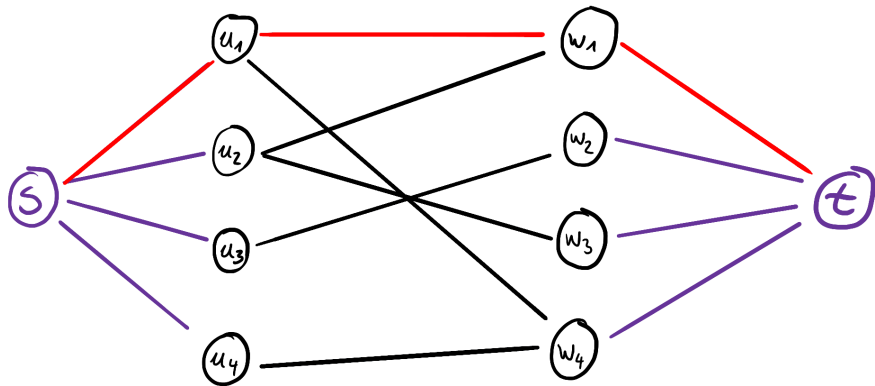


Also, every maximum flow will represent a maximum matching. Assume there is a matching that contains more $U - W$ -edges than the $U - W$ -edges of a maximum flow. Then one could construct a larger flow by using the $U - W$ -edges of the matching.

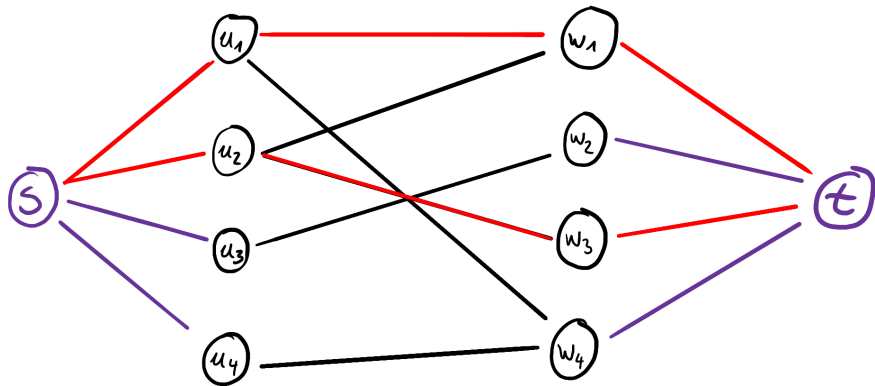
Matching via MaxFlow: example



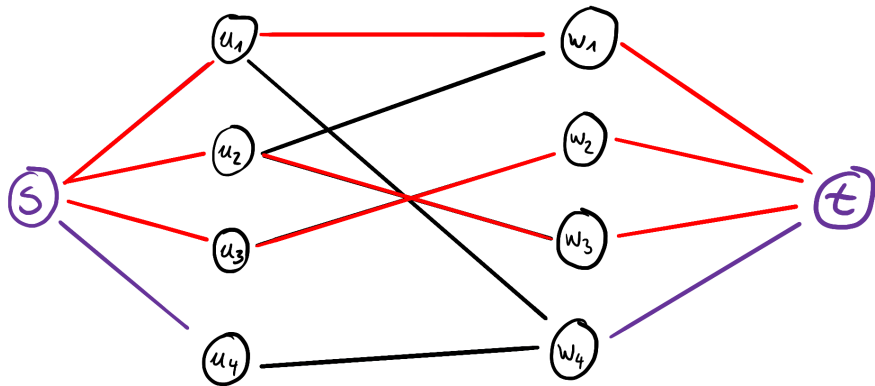
Matching via MaxFlow: example



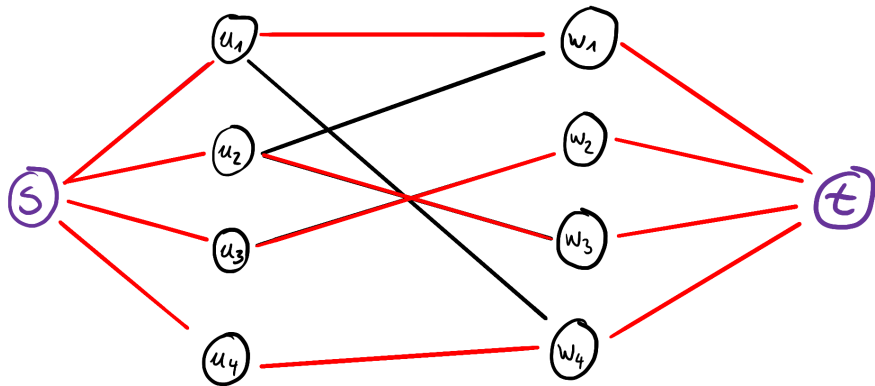
Matching via MaxFlow: example



Matching via MaxFlow: example



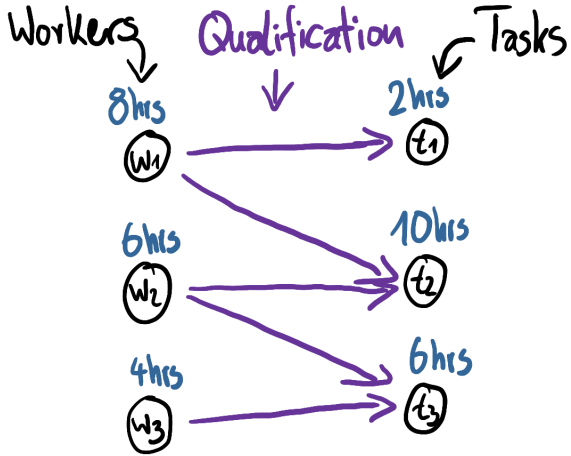
Matching via MaxFlow: example



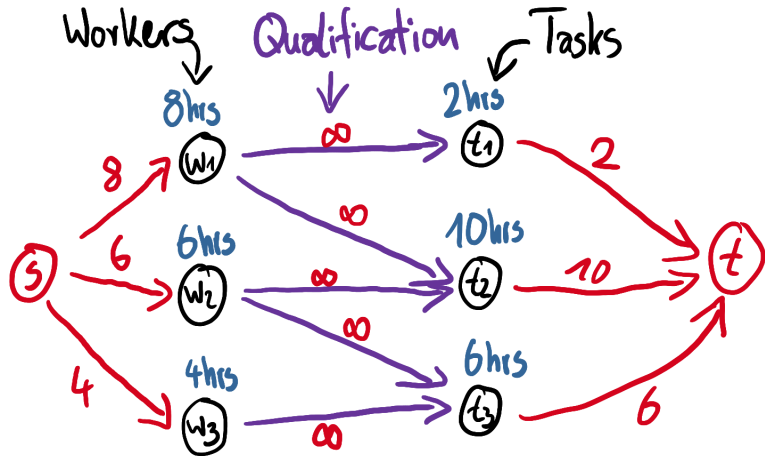
Variation of Matchings

- Consider a n -to- m -assignment of workers and tasks, that is: workers can work for different numbers of hours, and each task takes a different amount of time.
- Find an assignment such that all task get done by the workers (Note that each worker can work on multiple tasks and each task can be done by multiple workers).
- This is not a matching anymore, but it can be solved with a similar construction using a MaxFlow-algorithm.

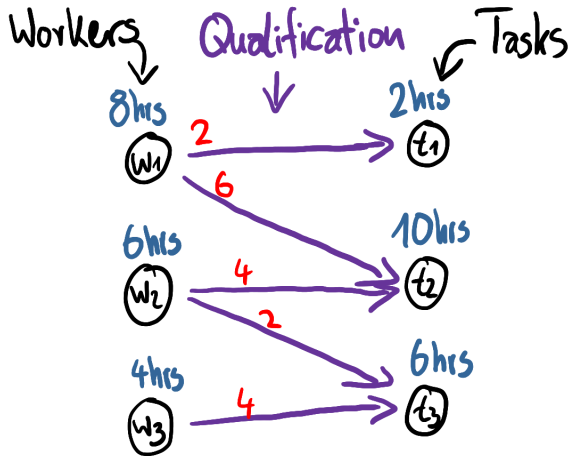
n-to-m-assignment



n-to-m-assignment



n-to-m-assignment



Interlude: Problems from Real Contests II

INTERLUDE: PROBLEMS FROM REAL CONTESTS II

- Same procedure as last week: again, we will look at a specific problem from the 2019 world final and try to construct an algorithm that solves it.
- We will not bother about the implementation in this example!
- At first, read the problem statement for yourself (next slide).

Problem A

Azulejos

Time limit: 10 seconds

Ceramic artists Maria and João are opening a small *azulejo* store in Porto. *Azulejos* are the beautiful ceramic tiles for which Portugal is famous. Maria and João want to create an attractive window display, but, due to limited space in their shop, they must arrange their tile samples in two rows on a single shelf. Each of João's tiles has exactly one of Maria's tiles in front of it and each of Maria's tiles has exactly one of João's tiles behind it. These hand-crafted tiles are of many different sizes, and it is important that each tile in the back row is taller than the tile in front of it so that both are visible to passers-by. For the convenience of shoppers, tiles in each row are arranged in non-decreasing order of price from left to right. Tiles of the same price may be arranged in any order subject to the visibility condition stated above.



Azulejo in the cathedral of Porto.
Source: [Wikimedia Commons](#)

Your task is to find an ordering of the tiles in each row that satisfies these constraints, or determine that no such ordering exists.

Input

The first line of input contains an integer n ($1 \leq n \leq 5 \cdot 10^5$), the number of tiles in each row. The next four lines contain n integers each; the first pair of lines represents the back row of tiles and the second pair of lines represents the front row. Tiles in each row are numbered from 1 to n according to their ordering in the input. The first line in each pair contains n integers $p_1, \dots, p_n$ ($1 \leq p_i \leq 10^9$ for each i), where p_i is the price of tile number i in that row. The second line in each pair contains n integers $h_1, \dots, h_n$ ($1 \leq h_i \leq 10^9$ for each i), where h_i is the height of tile number i in that row.

Output

If there is a valid ordering, output it as two lines of n integers, each consisting of a permutation of the tile numbers from 1 to n . The first line represents the ordering of the tiles in the back row and the second represents the ordering of the tiles in the front row. If more than one pair of permutations satisfies the constraints, any such pair will be accepted. If no ordering exists, output *impossible*.

Sample Input 1

```
4
3 2 1 2
2 3 4 3
2 1 2 1
2 2 1 3
```

Sample Output 1

```
3 2 4 1
4 2 1 3
```

Sample Input 2

```
2
1 2
2 3
2 8
2 1
```

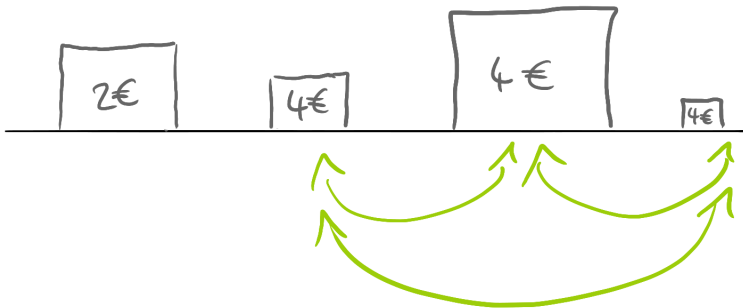
Sample Output 2

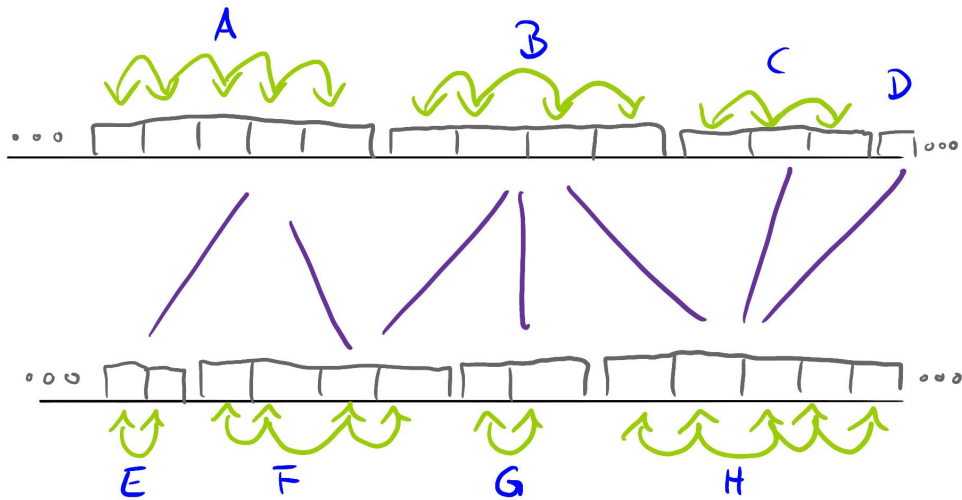
```
impossible
```

Back row
(João)



Front row
(Maria)





Solution:

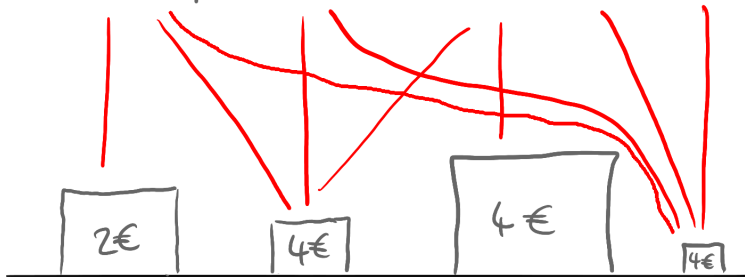
- construct a matching problem: match pairs of tiles that are placed in a common column.
 - $\mathcal{O}(|V|^2)$ to create flow-network and
 - (at least) $\mathcal{O}(|V|^3)$ for MaxFlow-algorithm

possible
matchings

Back row
(João)



Front row
(Maria)



Improvement: greedy approach that always matches tile from bottom row with smallest possible tile from top row.

Intuitively, this works because this approach will never destroy any possibilities for the remaining tiles.

- Standard matching:
 - $\mathcal{O}(|V|^2)$ to create flow-network and
 - (at least) $\mathcal{O}(|V|^3)$ for MaxFlow-algorithm.
- Greedy matching:
 - $\mathcal{O}(|V|^2)$ to create flow-network and
 - $\mathcal{O}(|E|)$ to find matching.

Exercise

Exercise

- The exercises for this week are more problems from `onlinejudge.org`.
- Solve 11506 – Angry Programmer.
- Solve 11167 – Monkeys in the Emei Mountain.
- Construct at least three additional test cases for each.
- Upload your code to your git repository.

The Problem setting is also included on the next two pages.

You, a programmer of an important software house, have been fired because you didn't solve an important problem that was assigned to you. You are very furious and want to take revenge on your boss, breaking the communication between his computer and the central server.

The computer of your boss and the central server are in the same network, which is composed of many machines (computers) and wires linking pairs of those machines. There is at most one wire between any pair of machines and there can be pairs of machines without a wire between them.

To accomplish your objective, you can destroy machines and wires, but you can't destroy neither the computer of your boss nor the central server, because those machines are monitored by security cameras. You have estimated the cost of blowing up each machine and the cost of cutting each wire in the network.

You want to determine the minimum cost of interrupting the communication between your boss' computer and the central server. Two computers A and B can communicate if there is a sequence of undestroyed machines $x_1, \dots, x_n$ such that $x_1 = A$, $x_n = B$ and x_i is linked with x_{i+1} with an uncut wire (for each $1 \leq i \leq n - 1$).

Input

The input consists of several test cases. Each test case is represented as follows:

- A line with two integers M and W ($2 \leq M \leq 50$, $0 \leq W \leq 1000$), representing (respectively) the number of machines and the number of wires in the network.
- $M - 2$ lines, one per machine (different from the boss' machine and the central server), containing the following information separated by spaces:
 - An integer i ($2 \leq i \leq M - 1$) with the identifier of the machine. Assume that the boss' machine has id 1 and that the central server has id M .
 - An integer c ($0 \leq c \leq 100000$) specifying the cost of destroying the machine.
- W lines, one per wire, containing the following information separated by spaces:
 - Two integers j and k ($1 \leq j < k \leq M$) specifying the identifiers of the machines linked by the wire. Remember that the wire is bidirectional.
 - An integer d ($0 \leq d \leq 100000$) specifying the cost of cutting the wire.

The end of the input is specified by a line with the string '0 0'.
Suppose that the machines have distinct identifiers.

Output

For each test case, print a line with the minimum cost of interrupting the communication between the computer of your boss and the central server.

Sample Input

4 4
3 5
2 2
1 2 3
1 3 3
2 4 1
3 4 3
4 4
3 2
2 2
1 2 3
1 3 3
2 4 1
3 4 3
0 0

Sample Output

4
3

Xuexue is a pretty monkey living in the Emei mountain. She is extremely thirsty during time 2 and time 9 everyday, so she must drink 2 units water during this period. She may drink water more than once, as long as the total amount of water she drinks is exactly 2 - she never drinks more than she needs. Xuexue drinks 1 unit water in one time unit, so she may drinks from time 2 to time 4, or from 3 to 5, ..., or from 7 to 9, or even drinks twice: first from 2 to 3, then from 8 to 9. But she can't drink from 1 to 3 since she's not thirsty at time 1, and she can't drink from 8 to 10, since she must finish at time 9.

There are many monkeys like Xuexue: we use a triple (v, a, b) to describe a monkey who is thirsty during time a and b , and must drink exactly v units of water during that period. Every monkey drinks at the same speed (i.e. one unit water per unit time).

Unfortunately, people keep on doing something bad on the environment in Emei Mountain. Eventually, there are only *one* unpolluted places for monkeys to drink. Further more, the place is so small that at most m monkeys can drink water together. Monkeys like to help each other, so they want to find a way to satisfy all the monkeys' need. Could you help them?

Input

The input consists of several test cases. Each case contains two integers n and m ($1 \leq n \leq 100, 1 \leq m \leq 5$), followed by n lines of three integer numbers (v, a, b) , where $0 \leq v, a, b \leq 50,000, a < b, 0 < v \leq b - a$. The last test case is followed by a single zero, which should not be processed. There are at most 50 test cases.

Output

For each test case, print the case number and whether there is a solution. If there is a solution, the following n lines describe the solution: one monkey on a separate line. The first number k means the monkey drinks water for k times. The following k pairs (a_i, b_i) means the monkey drinks from a_i to b_i ($a_i < b_i$). The pairs should be sorted in ascending order, and a_i should not be equal to a_{i+1} for $1 \leq i \leq k - 1$ (otherwise these two drinking periods could be combined). If more than one solution exists, any one is acceptable. Note that there should be exactly one space between k and pairs (a_i, b_i) , but no space within each pair.

Sample Input

3 1
2 2 9
2 3 5
3 5 8
2 1
4 5 9
4 8 12
5 2
2 1 3
2 3 5
2 5 7
2 1 7
4 2 6
0

Sample Output

Case 1: Yes
2 (2,3) (8,9)
1 (3,5)
1 (5,8)
Case 2: No
Case 3: Yes
1 (1,3)
1 (3,5)
1 (5,7)
2 (1,2) (6,7)
1 (2,6)

Exam Questions

- Describe the MinCut-Problem and give two examples with a small graph with 5 nodes and 7 edges (the graph has to be connected):
 - Construct your first example such that there is exactly one minimum cut, and at least one other cut that isn't a minimum cut.
 - Construct your second example such that there are at least three different minimum cuts, and at least one other cut that isn't a minimum cut.
- Describe the Matching-Problem and give two simple examples with a small graph with 6 nodes (that is a bipartite graph with 2×3 nodes):
 - Construct your first example such that there are two different perfect matchings.
 - Construct your second example such that no perfect matching is possible (but every node should have at least one possible partner!).